AWS ACM, SSL/TLS, and HTTPS

Complete Guide: From Certificates to Secure Production Architecture

Chapter 1: The Foundation - What is Security in Web Communication?

1.1 Why Secure Communication Matters

Every time a user visits your website, data travels across the internet between their browser and your server. This data passes through dozens of intermediate network devices including routers, switches, and internet service provider infrastructure. Without encryption, any device in this chain can read the data passing through it.

Consider what this means in practice. When a user enters their credit card number on your checkout page and clicks submit, that number travels from their browser across the internet to your server. Without encryption, anyone monitoring the network traffic at any point along that journey can read the credit card number in plain text. This includes malicious actors operating fake Wi-Fi hotspots in coffee shops, compromised routers at internet service providers, or government surveillance systems in certain countries.

The consequences of unencrypted web communication extend beyond credit card theft. Usernames and passwords sent without encryption can be captured and used to take over accounts. Personal health information transmitted without encryption violates privacy regulations. Business data sent without encryption can be intercepted by competitors. Even seemingly harmless data like browsing history can be used to build profiles for targeted attacks.

This is the fundamental problem that SSL, TLS, and HTTPS solve. They create an encrypted tunnel between the user's browser and your server so that even if a malicious actor captures the network traffic, they cannot read the content.

1.2 The Real-World Scenario That Drives This Guide

Imagine you are building a healthcare appointment booking platform called MediBook. Users enter their full names, dates of birth, insurance information, and medical history details. This is among the most sensitive data imaginable. Beyond the ethical obligation to protect this data, regulations like HIPAA in the United States mandate specific security requirements for healthcare data transmission.

You have built your application and deployed it on an EC2 instance. The application works perfectly. But when you visit it in a browser, you see a warning saying the connection is not secure. Your users see a red padlock icon and a frightening warning message telling them not to enter sensitive information. Potential users immediately leave without ever trying your service.

You need HTTPS. You need an SSL certificate. And you need to understand how all the pieces fit together so you can implement it correctly, maintain it without manual effort, and ensure it works reliably in production. This guide takes you through every concept and every step.

Chapter 2: Understanding SSL, TLS, and HTTPS

2.1 The Terminology Confusion Explained

When people discuss secure web communication, they use several terms interchangeably, which creates significant confusion. Before going any further, it is worth clearly defining each term and explaining how they relate to each other.

SSL stands for Secure Sockets Layer. SSL was the original protocol developed by Netscape in the mid-1990s to create encrypted connections between web browsers and servers. SSL went through several versions, with SSL 3.0 being the last version before it was replaced. SSL is now considered cryptographically broken and insecure. It should not be used in any modern system.

TLS stands for Transport Layer Security. TLS is the modern, secure successor to SSL. TLS 1.0 was released in 1999, TLS 1.1 in 2006, TLS 1.2 in 2008, and TLS 1.3, the current version, in 2018. TLS 1.3 introduced significant security improvements and performance optimizations over previous versions. When people talk about securing web connections today, TLS is what they are actually using. Despite SSL being deprecated and replaced by TLS, the industry still uses the term SSL colloquially. When your hosting provider talks about getting an SSL certificate, they mean a TLS certificate. When AWS Certificate Manager says it issues SSL/TLS certificates, the certificates are TLS certificates. When a browser shows a padlock icon indicating a secure connection, it is using TLS. You will see both terms used throughout this guide and throughout the industry. They refer to the same concept in modern usage, and the actual protocol in use is always TLS.

HTTPS stands for Hypertext Transfer Protocol Secure. HTTP is the protocol used by browsers to request web pages and by servers to serve them. HTTPS is simply HTTP with TLS encryption applied. When a website uses HTTPS, all HTTP communication between the browser and server is encrypted using TLS. The S in HTTPS literally means secure.

2.2 How TLS Encryption Works - The Conceptual Model

Understanding TLS at a conceptual level helps you make better architectural decisions and debug problems when they occur. You do not need to understand the deep mathematics, but understanding the key concepts is valuable.

The Problem TLS Solves

TLS needs to solve two distinct problems simultaneously. The first problem is encryption, meaning how do two parties communicate privately when their messages travel across a public network that anyone can monitor. The second problem is authentication, meaning how does the browser know it is

actually talking to the real MediBook server and not an attacker pretending to be MediBook.

Symmetric vs Asymmetric Encryption

TLS uses two types of encryption that work together. Symmetric encryption uses the same key to both encrypt and decrypt data. If you and a friend both have an identical physical key and a lockbox, you can put messages in the lockbox, lock it, and only your friend with the identical key can unlock it to read the message. Symmetric encryption is extremely fast and efficient, which is why it is used for encrypting the actual data in a TLS connection. The challenge is that both parties need the same key, and you need a secure way to share that key in the first place.

Asymmetric encryption, also called public key cryptography, uses a mathematically related pair of keys. One key is the public key, which can be freely shared with anyone. The other key is the private key, which must be kept completely secret. Data encrypted with the public key can only be decrypted with the corresponding private key. Data signed with the private key can be verified by anyone with the public key. Asymmetric encryption is much slower than symmetric encryption, so it is not practical for encrypting large amounts of data. However, it elegantly solves the key exchange problem.

TLS uses asymmetric encryption to securely establish a shared symmetric key at the beginning of the connection. Once both parties have the shared symmetric key, they switch to the much faster symmetric encryption for all subsequent data transfer. This combination gives TLS both strong security and good performance.

The Role of Certificates

Here is where digital certificates become essential. Asymmetric encryption alone does not prevent a man-in-the-middle attack. Suppose an attacker intercepts your connection to MediBook. The attacker generates their own key pair and sends you their public key, pretending it is MediBook's public key. You encrypt data with the attacker's public key, thinking it is MediBook's. The attacker decrypts your data with their private key, reads it, then re-encrypts it with the real MediBook public key and forwards it. Both you and MediBook think you are talking directly to each other, but the attacker has read everything.

Digital certificates solve this by providing authentication. A certificate is a digital document that contains the server's public key along with information about who owns that key, specifically the domain name. The certificate is digitally signed by a trusted Certificate Authority.

A Certificate Authority is an organization that is trusted by browsers to verify that a public key actually belongs to the entity claiming to own a specific domain name. Before issuing a certificate for MediBook's domain, the Certificate Authority verifies that the entity requesting the certificate actually controls that domain. The Certificate Authority then signs the certificate with its own private key.

Browsers come pre-installed with the public keys of hundreds of trusted Certificate Authorities. When a browser receives a certificate from a server, it checks whether the certificate was signed by one of these trusted Certificate Authorities. If it was, the browser knows that the Certificate Authority has verified that this public key belongs to the owner of this domain. The browser can then trust that it is communicating with the legitimate server.

AWS Certificate Manager acts as a Certificate Authority, or works with Certificate Authorities, to issue and manage these certificates for your domains. When ACM issues a certificate for your domain, browsers trust it because ACM's certificates are backed by trusted Certificate Authorities that are already trusted by all major browsers.

2.3 The TLS Handshake Step by Step

When a browser connects to your HTTPS website, a process called the TLS handshake occurs before any actual application data is exchanged. The browser opens a connection to your server and sends a Client Hello message. This message contains the TLS version the browser supports, a list of cipher suites the browser can use, and a random number that will be used later in the key derivation process. The server responds with a Server Hello message. This message contains the TLS version and cipher suite the server has selected from the browser's list, and another random number. The server then sends its digital certificate. This certificate contains the server's public key and the domain name it is valid for, signed by a Certificate Authority. The browser validates the certificate. It checks that the certificate was signed by a trusted Certificate Authority. It checks that the domain name in the certificate matches the domain the browser is connecting to. It checks that the certificate has not expired. It checks that the certificate has not been revoked. If all validations pass, the browser and server use the exchanged

random numbers and the server's public key to establish a shared symmetric encryption key. From this point forward, all communication between the browser and server is encrypted with this shared symmetric key. The TLS handshake is complete, and the connection is secure. The entire TLS handshake happens in milliseconds and is completely invisible to users. All they see is the padlock icon in their browser indicating that the connection is secure.

2.4 What HTTPS Protects and What It Does Not

It is important to understand both what HTTPS protects and where its limitations lie. HTTPS protects data in transit. It encrypts the data traveling between the browser and the server, preventing interception and reading of that data by third parties. It verifies that the server is who it claims to be, preventing man-in-the-middle attacks. It also ensures data integrity, meaning it detects if anyone has tampered with the data in transit. HTTPS does not protect data at rest on your server. If your database is compromised, HTTPS offers no protection. It does not protect against vulnerabilities in your application code, such as SQL injection or cross-site scripting. It does not protect against a malicious server, because HTTPS only verifies that you are talking to the legitimate owner of the domain, not that the owner has good intentions. It does not protect against malware on the user's device.

For the MediBook healthcare platform, HTTPS is a necessary component of security, but it is one layer among many. The data also needs to be encrypted at rest in the database, access controls need to restrict who can view patient data, application code needs to be secure against common vulnerabilities, and audit logs need to track all access to sensitive data.

Chapter 3: AWS Certificate Manager (ACM)

3.1 What is AWS Certificate Manager?

AWS Certificate Manager, commonly abbreviated as ACM, is a fully managed service that handles the provisioning, management, deployment, and renewal of SSL/TLS certificates for use with AWS services. ACM eliminates the

time-consuming manual processes of purchasing, uploading, and renewing SSL certificates. Before services like ACM existed, obtaining and managing SSL certificates was a genuinely painful process. You would generate a Certificate Signing Request on your server, submit it to a Certificate Authority, pay an annual fee often ranging from tens to hundreds of dollars, wait for validation to complete, download the certificate files, upload them to your server, configure your web server to use them, and then repeat this entire process every year when the certificate expired. If you forgot to renew, your certificate would expire and users would see terrifying security warnings. ACM replaces this entire process with a streamlined, mostly automated workflow. You request a certificate through the ACM console or API, complete a one-time domain validation, and ACM issues the certificate. When the certificate is near expiration, ACM automatically renews it without any action required from you. For certificates attached to AWS services like Application Load Balancers and CloudFront distributions, the renewal and deployment are entirely automatic.

3.2 What ACM Can and Cannot Do

ACM can issue public SSL/TLS certificates for domains you own, completely free of charge. It can issue private certificates for internal services using AWS Private Certificate Authority, which is a paid service. It can automatically renew certificates it has issued before they expire. It can deploy certificates directly to supported AWS services including Application Load Balancers, Network Load Balancers, CloudFront distributions, API Gateway, Elastic Beanstalk environments, and AWS Nitro Enclaves. It can import certificates that you have obtained from external Certificate Authorities. ACM cannot install certificates directly on EC2 instances. This is the most important limitation to understand. The private key of an ACM-managed certificate never leaves AWS infrastructure. AWS stores the private key securely and uses it when your load balancer or CloudFront distribution performs TLS operations. Because the private key cannot be exported, you cannot download an ACM certificate and install it on your EC2 instance directly.

3.3 ACM Certificate Types

ACM offers two fundamental types of certificates based on how they are validated. Domain Validated certificates, commonly called DV certificates, verify only that you control the domain name. No information about your organization is included in the certificate beyond the domain name. Domain Validated certificates are the standard certificate type for web applications

and APIs. They provide encryption and authentication of the domain, which is what most applications need. Organization Validated and Extended Validation certificates include verified information about your organization. They require the Certificate Authority to verify your organization's legal existence and identity, which involves more extensive vetting. ACM does not issue Organization Validated or Extended Validation certificates. If your application requires these certificate types, you must obtain them from an external Certificate Authority and import them into ACM.

Chapter 4: Amazon Route 53 - DNS for AWS Applications

Amazon Route 53 is AWS's highly available and scalable Domain Name System service. DNS is the system that translates human-readable domain names like www.medibook.com into the IP addresses that computers use to route network traffic. Route 53 is relevant to SSL and ACM in two important ways: DNS validation for ACM certificates, and pointing domain names to AWS resources like load balancers. Route 53 core concepts include Hosted Zones (a container for DNS records). Relevant record types include A record (IP address), CNAME record (domain to domain), and Alias record (domain to AWS resource). Application Load Balancers do not have static IP addresses. If you used standard A records, they might become invalid. Alias records resolve to an AWS resource, so when load balancer IP addresses change, Route 53 automatically reflects those changes. Alias records can be used at the zone apex, unlike CNAME records.

Chapter 5: SSL Termination - Where Decryption Happens

SSL termination refers to the point where encrypted TLS traffic is decrypted and processed as plain HTTP. SSL termination at the load balancer is the standard, recommended practice. The certificate lives on the load balancer, which handles decryption and forwards plain HTTP over the private VPC network to EC2. It requires no SSL configuration on the EC2 instance itself. End-to-end encryption means traffic stays encrypted from browser to

instance. This increases compliance but significantly increases complexity as servers must manage their own certificates. The ALB adds X-Forwarded-Proto headers to reveal the original protocol (http/https) and X-Forwarded-For for the original client IP to the internal application, as these would otherwise be obscured by the ALB.

Chapter 6: The Complete HTTPS Architecture on AWS

Route 53 directs traffic to the load balancer, which uses the ACM certificate to terminate SSL, then forwards plain HTTP traffic to instances within the private VPC. Request flow is Request -> Route 53 (DNS) -> ALB (TLS termination) -> EC2 (Plain HTTP) -> Response back through ALB (Encrypted). You should configure a 301 redirect rule on the ALB's port 80 listener to point to port 443.

Chapter 7: Setting Up ACM and HTTPS - Complete Walkthrough

Prerequisites include domain name, DNS control, and AWS account. You need to create a Hosted Zone in Route 53 and update registrar NS records. Navigate to ACM, request a certificate, add root and wildcard domain names, choose DNS validation. Create CNAME records in Route 53. Status changes to Issued. For the ALB, name ALB, select subnets (Multi-AZ), create security group, add port 443 listener with certificate, add port 80 listener with 301 redirect. Register EC2 instances with the defined Target Group. Create Alias A records for your root domain and www subdomain pointing to the ALB.

Remaining Chapters Summary

Chapter 8 (ACM Renewal): 13-month validity; renewal begins 60 days before expiration. DNS validation enables automatic renewal.

Chapter 9 (CloudFront): For global presence, reduce latency, edge caching, WAF integration.

Chapter 10 (ACM vs Let's Encrypt): Let's Encrypt = Certificate on Instance. ACM = Certificate on AWS Managed Service.

Chapter 11 (Case Studies): MediBook (Used ALB+ACM for HIPAA), GlobalShop (Used CloudFront+ACM for global latency), DevConnect (Used wildcard certs for subdomains).

Chapter 12 (Security Best Practices): Enforce TLS 1.2 or 1.3, use HSTS, restrict direct EC2 access, monitor certificate events.